C++11/14 多线程与并发编程通关指南（面试级）

本文档按照 **理解难度 + 面试频率** 科学编排，涵盖从基础 API 到操作系统底层的全栈并发知识体系。适用于 C++ 后端开发、音视频、游戏引擎及量化交易等高并发岗位的面试突击与体系化学习。

🗺️ 第一部分：C++11/14 并发编程知识全景地图

```
                          C++11/14 并发编程体系
                                  |
  ┌──────────────────────────────────────────────┼──────────────────────────────
  |                               |                                |
[线程管理]                      [线程同步]                       [线程通信]
(Thread)                         (Mutex)                          (Condition)
  |                               |                                |
std::thread                     std::mutex                       std::condition_variable
  ├── join()                        ├── lock()                       ├── wait()
  ├── detach()                      ├── unlock()                     ├── notify_one(
  └── joinable()                    └── try_lock()                   └── notify_all(
  |                               |                                |
[命名空间]                       [RAII 锁管理]                    [等待机制]
std::this_thread                std::lock_guard                  wait_for()
  ├── get_id()                   std::unique_lock                 wait_until()
  ├── sleep_for()                std::scoped_lock (C++17)         |
  └── yield()                       |                              └── [核心] 谓词(P
                                  |
                        ┌──────────┴──────────┐
                        |                     |
                  [高级锁类型]            [单次初始化]
                  shared_mutex(C++14)     std::call_once
                  recursive_mutex         std::once_flag

  ┌──────────────────────────────────────────────┼──────────────────────────────
  |                               |                                |
[无锁并发]                      [异步任务模型]                   [时间与内存模型]
(Atomic)                         (Async & Future)                 (Chrono & Memory)
  |                               |                                |
std::atomic<T>                  std::async                       std::chrono
  ├── load()                        |                                ├── duration
  ├── store()                    std::future                         ├── time_point
  ├── fetch_add()                   ├── get()                        └── steady_cloc
  └── CAS操作                       └── wait()                       |
     ├── compare_exchange_weak()    |                              [C++ 内存模型]
     └── compare_exchange_strong()  |                              std::memory_order
                                 std::promise                        ├── memory_order_s
                                 std::packaged_task                  ├── memory_order_c
                                                                     ├── memory_order_r
                                                                     └── memory_order_r
```

14只进入了带time，无time的在17引入

🎯 第二部分：按图索骥 —— 12 阶段攻克清单

建议在学习时，每掌握一个知识点就在心里打个勾。

阶段 1：线程基础（入门必修）

- [ ] **std::thread 基本使用**：创建线程、传递参数（注意传引用需用 `std::ref`）。
- [ ] **生命周期管理**：`join()` 阻塞等待，`detach()` 分离后台运行，`joinable()` 检查状态。
- [ ] **对象特性**：`std::thread` 禁用了拷贝构造（Cannot copy），只能移动（`std::move`）。
- [ ] **线程信息与控制**：`std::this_thread::get_id()`，`sleep_for()`，`yield()`（让出CPU时间片）。

阶段 2：互斥锁（并发安全基础）

- [ ] **核心概念**：临界区（Critical Section）、竞态条件（Race Condition）。
- [ ] **基础锁**：`std::mutex` 的 `lock()` 与 `unlock()`。
- [ ] **RAII 加锁（重点）**：
  - `std::lock_guard`：轻量级，作用域内自动加解锁，不可手动控制。
  - `std::unique_lock`：重量级，支持延迟加锁（`std::defer_lock`）、尝试加锁、配合条件变量使用。

阶段 3：死锁问题（Bug 终结者）

- [ ] **死锁四要素**：互斥、占有且等待、不可抢占、循环等待。
- [ ] **经典场景**：线程A锁1求2，线程B锁2求1 (A->B, B->A)。
- [ ] **解决手段**：
  - 固定加锁顺序。
  - 使用 `std::lock(m1, m2, ...)` 或 `std::scoped_lock` (C++17) 一次性锁住多个互斥量。

阶段 4：条件变量（线程间通信）

- [ ] **核心 API**：`std::condition_variable`，`wait()`，`notify_one()`，`notify_all()`。
- [ ] **面试拷问**：
  - 为什么 `wait` 必须配合 `unique_lock`？（因为 wait 内部需要多次解锁和重新加锁）。
  - 什么是**虚假唤醒（Spurious Wakeup）**？如何用 `wait(lock, []{ return condition; })` 的谓词解决?

阶段 5：原子操作（无锁并发进阶）

- [ ] **基础操作**：`std::atomic<T>`，`load()`，`store()`，`fetch_add()`。
- [ ] **CAS 操作（核心）**：`compare_exchange_weak` vs `compare_exchange_strong`（解决 ABA 问题与自旋锁实现）。
- [ ] **面试拷问**：为什么 `i++` 不是线程安全的？（读、改、写三步操作非原子性）。

阶段 6-8：异步任务与结果传递

- [ ] **std::async & std::future**：比 thread 更高级，直接获取子线程返回值（`future.get()`）。
- [ ] **std::promise**：在一个线程中计算，在另一个线程中通过 future 提取结果（`promise.set_value()`）。
- [ ] **std::packaged_task**：将普通函数或 Callable 包装为可异步执行的任务。

阶段 9-10：初始化与时间库

- [ ] **单次初始化**：`std::call_once` 与 `std::once_flag`（实现完美线程安全单例的利器）。
- [ ] **时间库应用**：`std::chrono::steady_clock`（适合计算时间差），`sleep_for(chrono::milliseconds(100))`。

阶段 11：内存模型（资深/专家岗必问）

- [ ] **概念剖析**：Data Race vs Race Condition，Happens-before 语义。
- [ ] **Memory Order**：
  - `seq_cst`（顺序一致性，默认，开销最大）。
  - `acquire` / `release`（获取/释放语义，常用于构建同步锁）。
  - `relaxed`（松散模型，只保证单一变量的原子性，适合计数器）。

🚀 第三部分：高级附加题 —— C++多线程底层运行原理

> **面试官想听的不仅仅是你会用 API，而是你懂得“这行代码在 CPU 和 OS 层面发生了什么”。**

1. CPU 缓存一致性与 False Sharing（伪共享）

- **物理真相**：现代 CPU 有 L1/L2/L3 缓存。CPU 读写内存不是按字节，而是按 **Cache Line（缓存行，通常为 64 Bytes）**。
- **MESI 协议**：为了保证多核缓存数据一致性，CPU 硬件实现了 MESI（修改、独占、共享、失效）协议。
- **伪共享 (False Sharing) Bug**：
  - **现象**：线程 A 修改变量 `x`，线程 B 修改变量 `y`。逻辑上不冲突，但如果 `x` 和 `y` 在物理内存上挨得太近（处于同一个 64 Bytes 的 Cache Line 内），A 修改 `x` 会导致整个 Cache Line 失效，B 必须重新从主存拉取数据，导致性能急剧下降。
  - **解决方案**：空间换时间。使用 `alignas(64)` 强制内存对齐，让互相独立的高频并发变量分处不同的 Cache Line。

2. Mutex（互斥锁）的底层代价

- **OS 视角**：C++ 的 `std::mutex` 在 Linux 下通常基于 `pthread_mutex`，底层是 **Futex (Fast Userspace Mutex)**。
- **运行机制**：

    - **无冲突时**：纯用户态原子操作（快）。
    - **发生冲突时**：调用系统调用（如 `futex_wait`），当前线程陷入内核态，由操作系统将其挂起（休眠），引发 **上下文切换 (Context Switch)**。
- **代价**：上下文切换需要保存寄存器、程序计数器，甚至刷新 TLB（TLB Flush），开销极大（通常微秒级）。

3. Atomic（原子操作）的底层魔法

- **硬件支持**：`std::atomic` 底层直接映射为 CPU 指令（如 x86 的 `LOCK CMPXCHG`）。
- **锁总线/锁缓存**：带有 `LOCK` 前缀的汇编指令会锁定内存总线或相关的 Cache Line，确保指令执行时不受其他核心干扰。
- **对比 Mutex**：无内核态切换，无线程挂起，是"忙等待（Spinning）"模型。适合临界区极短的代码；若临界区长，自旋反而会榨干 CPU 资源。

4. Memory Order（内存屏障/指令重排）

- **为什么需要？**：为了极致性能，**编译器**（编译期）和 **CPU**（运行期）都会偷偷打乱你的代码执行顺序（Out-of-Order Execution）。在单线程下没问题，但在多线程下会导致灾难（如：初始化没完成，指针就先暴露出去了）。
- **本质**：Memory Order 底层插入的是 **内存屏障（Memory Barrier/Fence）** 指令，强制要求 CPU 必须按既定顺序刷出 Store Buffer 或执行 Load。

💻 第四部分：必须手撕的经典代码模板

1. 两个线程交替打印奇偶数 (Condition Variable)

考察点：条件变量的谓词、互斥锁、虚假唤醒。

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>

std::mutex mtx;
std::condition_variable cv;
int counter = 1;
const int MAX = 10;

void printOdd() {
    while (counter <= MAX) {
        std::unique_lock<std::mutex> lock(mtx);
        cv.wait(lock, []() { return counter % 2 == 1; });
        if (counter > MAX) break;
        std::cout << "Odd: " << counter++ << std::endl;
        cv.notify_one();
    }
}

void printEven() {
    while (counter <= MAX) {
        std::unique_lock<std::mutex> lock(mtx);
```

```
        cv.wait(lock, []() { return counter % 2 == 0; });
        if (counter > MAX) break;
        std::cout << "Even: " << counter++ << std::endl;
        cv.notify_one();
    }
}

int main() {
    std::thread t1(printOdd);
    std::thread t2(printEven);
    t1.join();
    t2.join();
    return 0;
}
```

2. 线程安全单例模式 (Thread-safe Singleton)

考察点： `std::call_once` 或 C++11 局部静态变量特性。

推荐写法一（C++11 局部静态变量，最简洁，又称 Meyers Singleton）：

```
class Singleton {
public:
    static Singleton& getInstance() {
        // C++11 标准保证局部静态变量的初始化是线程安全的
        static Singleton instance;
        return instance;
    }
private:
    Singleton() = default;
    Singleton(const Singleton&) = delete;
    Singleton& operator=(const Singleton&) = delete;
};
```

写法二（使用 std::call_once，适合更复杂的延迟初始化）：

```
#include <mutex>

class Singleton {
public:
    static Singleton* getInstance() {
        std::call_once(initFlag, []() {
            instance = new Singleton();
        });
        return instance;
    }
private:
    Singleton() = default;
    static Singleton* instance;
    static std::once_flag initFlag;
};
Singleton* Singleton::instance = nullptr;
```

```
std::once_flag Singleton::initFlag;
```

3. 多线程计数器 (Atomic)

考察点：锁的开销 vs 无锁编程。

```
#include <atomic>
#include <thread>
#include <vector>

std::atomic<int> global_counter(0); // 替换掉原先的 std::mutex + int

void worker() {
    for (int i = 0; i < 10000; ++i) {
        // fetch_add 是原子操作，无需加锁，性能远超 mutex
        global_counter.fetch_add(1, std::memory_order_relaxed);
    }
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 10; ++i) threads.emplace_back(worker);
    for (auto& t : threads) t.join();
    // 最终一定输出 100000
}
```

💡 **总结与建议** 对于准备面试的开发者：

1. **基础打牢**：把前 6 个核心模块（Thread, Mutex, Condition Variable, Lock_guard, Unique_lock, Atomic）背到肌肉记忆。